Implémentations massivement parallèles de l'évolution différentielle et de l'optimisation par essaim particulaire sur GPU (CUDA)

Kossi Noumagno
Master 2 IMDS
Université de Haute-Alsace
Mulhouse, France
noumagnokossi0@gmail.com

Marieme Mbengue
Master 2 IMDS
Université de Haute-Alsace
Mulhouse, France
mariemembengue720@gmail.com

Rayhan Saouir
Master 2 IMDS
Université de Haute-Alsace
Mulhouse, France
srayhan2018@gmail.com

***Résumé*—L'évolution différentielle (DE) et l'optimisation par essaim particulaire (PSO) manipulent une population dont tous les membres subissent les mêmes opérations : ce sont des candidats naturels pour le calcul sur GPU. Pourtant, un portage CUDA mal pensé peut être plus lent que le code séquentiel. Dans ce travail, nous étudions cinq implémentations : un PSO séquentiel (PSO-CPU) et un PSO CUDA à quatre kernels (PSO-GPU), qui constituent notre point de départ, puis trois versions de DE/rand/1/bin : une version séquentielle (DE-CPU), une version CUDA de base à six kernels (DE-GPU) et une version améliorée inspirée de cudaDE$_i$ (DE-GPU-OPT), qui fusionne les kernels, garde les vecteurs d'essai en mémoire partagée, recouvre la préparation des indices de mutation grâce à deux flux et choisit la taille de bloc à l'exécution. Les cinq programmes sont évalués sur quatre fonctions décalées de CEC2005 (Sphere, Rosenbrock, Griewank, Rastrigin) pour $D \in \{10, 50, 100\}$, $P \in \{50, 100, 500\}$ et $\mathrm{MaxFEs} = 10^4 D$, avec 10 exécutions par cas, sur une NVIDIA Tesla T4. DE-GPU-OPT est de 2,4 à 33 fois plus rapide que DE-CPU et de 1,35 à 2,9 fois plus rapide que DE-GPU, alors que DE-GPU est plus lent que le CPU sur le plus petit problème ($0{,}91\times$) ; PSO-GPU atteint au plus $9{,}4\times$. Les trois versions de DE donnent des erreurs finales statistiquement indiscernables, et DE est nettement meilleur que PSO, sauf sur Rastrigin en grande dimension. Nous comparons aussi nos résultats à ceux de l'article de référence et discutons les écarts observés.**

***Index Terms*—CUDA, GPU, évolution différentielle, optimisation par essaim particulaire, calcul massivement parallèle, fusion de kernels, flux CUDA, CEC2005.**

I. Introduction

Les algorithmes évolutionnaires font évoluer une population de solutions candidates grâce à des opérateurs de sélection, de reproduction et de remplacement. Comme tous les individus subissent les mêmes opérations, ces algorithmes sont parallèles en données. Les GPU, qui regroupent des centaines de cœurs simples, semblent donc bien adaptés à ce type de calcul. Il faut toutefois se méfier : l'efficacité d'un programme CUDA dépend de la quantité de données échangées via la mémoire globale, de la configuration de lancement des kernels et de l'occupation des multiprocesseurs. Sans ces précautions, un programme parallèle peut être plus lent que son équivalent séquentiel [1].

L'évolution différentielle (DE) [2] est l'une des métaheuristiques les plus efficaces pour l'optimisation continue et sa structure se prête bien à la parallélisation. Qin *et al.* [1] ont montré que les implémentations CUDA existantes souffraient de deux défauts : un trafic excessif vers la mémoire globale et une mauvaise utilisation du GPU. Ils ont proposé cudaDE$_i$, qui fusionne des kernels, détermine automatiquement la configuration de lancement et utilise des flux pour recouvrir des kernels.

Ce travail a été réalisé dans le cadre d'un projet de Master. Le point de départ est un code CUDA de PSO fourni par l'enseignant. Notre démarche a été la suivante : (i) comprendre et valider ce code, (ii) le transformer en DE/rand/1/bin selon l'article [1], en gardant le PSO comme second algorithme étudié, (iii) évaluer l'ensemble avec le protocole du projet et (iv) comparer nos résultats à ceux de l'article.

a) Contributions.: (1) Une base de code unique, constituée de quatre fichiers (`kernel.h`, `kernel.cpp`, `kernel.cu`, `main.cpp`), qui contient cinq architectures : DE-CPU, DE-GPU, DE-GPU-OPT, PSO-CPU et PSO-GPU. Elles partagent les mêmes fonctions de test, les mêmes vecteurs de décalage, le même budget d'évaluations et les mêmes graines. (2) Une évaluation selon le protocole du projet (4 fonctions, 3 dimensions, 3 tailles de population, 10 exécutions), soit 1 800 exécutions, avec des tests statistiques. (3) Une règle de choix de la taille de bloc qui équilibre la charge entre les SM, que nous avons ajoutée à la conception de cudaDE$_i$. (4) Une comparaison avec les résultats de l'article de référence, avec une discussion des écarts que nous n'avons pas pu expliquer entièrement.

b) Organisation.: La section II rappelle les notions CUDA utiles. La section III présente le DE séquentiel et ses versions massivement parallèles (CUDA de base et améliorée). La section IV présente le PSO séquentiel et sa version CUDA. La section V présente les expériences et les résultats, et la

section VI conclut.

II. Programmation GPU avec CUDA

Un GPU est organisé en multiprocesseurs de flux (SM). Chaque SM contient de nombreux cœurs, des registres et une petite mémoire partagée rapide, tandis que tous les SM accèdent à une grande mémoire globale, plus lente [6]. Un *kernel* est une fonction exécutée par une grille de blocs de threads. Un bloc tourne sur un seul SM et ses threads, regroupés par *warps* de 32, peuvent coopérer par la mémoire partagée et des barrières. Des blocs différents ne communiquent que par la mémoire globale. Deux kernels successifs échangent donc leurs résultats par la mémoire globale, et chaque lancement de kernel a un coût fixe de l'ordre de quelques microsecondes.

Nous avons retenu trois points qui déterminent les performances d'un algorithme évolutionnaire sur GPU :

- **le trafic en mémoire globale** : chaque opérateur écrit dans un kernel séparé puis relu par le suivant, ce qui coûte cher ;
- **l'occupation** : le nombre de threads actifs par SM est limité par les registres, la mémoire partagée et le nombre de blocs, et la configuration de lancement doit être adaptée à (P, D) ;
- **la concurrence** : des kernels indépendants peuvent s'exécuter en même temps dans des flux (*streams*) différents.

Les nombres aléatoires sont générés sur le GPU avec cuRAND (générateur Philox), avec un état par thread.

a) Évaluation de la fonction objectif.: Pour que le CPU et le GPU utilisent exactement les mêmes équations, la fitness est écrite une seule fois dans `kernel.h`, avec deux fonctions `host/device`. La première, `fit_term`, calcule pour chaque coordonnée j un terme a_j (somme) et un terme b_j (produit, utilisé seulement par Griewank). La seconde, `fit_final`, combine ces termes et ajoute le biais. Le CPU parcourt les coordonnées dans une boucle, alors que le GPU confie une coordonnée à chaque thread et réduit les résultats en arbre dans la mémoire partagée. La fitness est toujours accumulée en double précision : avec un biais de -450, une erreur de 10^{-8} n'est pas représentable en simple précision.

III. Évolution différentielle

A. Aperçu et version séquentielle (DE-CPU)

Le DE fait évoluer une population $\{\mathbf{X}_i\}_{i=1}^{P}$ avec trois opérations successives. Pour chaque vecteur cible $\mathbf{X}_i$, DE/rand/1 produit un vecteur mutant

$$\mathbf{V}_i = \mathbf{X}_{r_1} + F\,(\mathbf{X}_{r_2} - \mathbf{X}_{r_3}), \tag{1}$$

où r_1, r_2, r_3 et i sont deux à deux distincts. Le croisement binomial donne le vecteur d'essai $\mathbf{U}_i$:

$$u_{i,j} = \begin{cases} v_{i,j} & \text{si } \mathrm{rand}_j(0,1) \le CR \text{ ou } j = j_{rand}, \\ x_{i,j} & \text{sinon,} \end{cases} \tag{2}$$

où j_{rand} garantit qu'au moins une coordonnée vient du mutant. Enfin, le remplacement est glouton : $\mathbf{U}_i$ remplace $\mathbf{X}_i$ si $f(\mathbf{U}_i) < f(\mathbf{X}_i)$. Nous utilisons $F = 0{,}5$ et $CR = 0{,}3$, comme l'article de référence [1].

Algorithm 1 DE/rand/1/bin séquentiel (DE-CPU)

```
initialiser X ~ U(lo, hi) ; évaluer fit[i] = f(X_i)
for g = 1 à G do
    for i = 1 à P do
        tirer r1 ≠ r2 ≠ r3 ≠ i et j_rand
        U ← mutation (1) et croisement (2)
        X_i^next ← U si f(U) < fit[i], sinon X_i
    end for
    échanger(X, X^next)
end for
return meilleur individu
```

Le budget est fixé en nombre d'évaluations, $\mathrm{MaxFEs} = 10^4 D$. Chaque génération coûte P évaluations, donc le nombre de générations est

$$G = \lfloor \mathrm{MaxFEs}/P \rfloor - 1. \tag{3}$$

Comme G diminue quand P augmente, une grande population ne fait que peu de générations : c'est ce qui explique plusieurs résultats de la section V. Par exemple, pour $D = 10$, on a $G = 1999$ pour $P = 50$ et $G = 199$ pour $P = 500$.

La population est stockée par individu ($X[i \cdot D + j]$) et double-bufferisée : une génération lit seulement l'ancienne population. La mise à jour est donc synchrone, comme dans les deux versions CUDA, ce qui rend la comparaison équitable.

B. Description des versions massivement parallèles

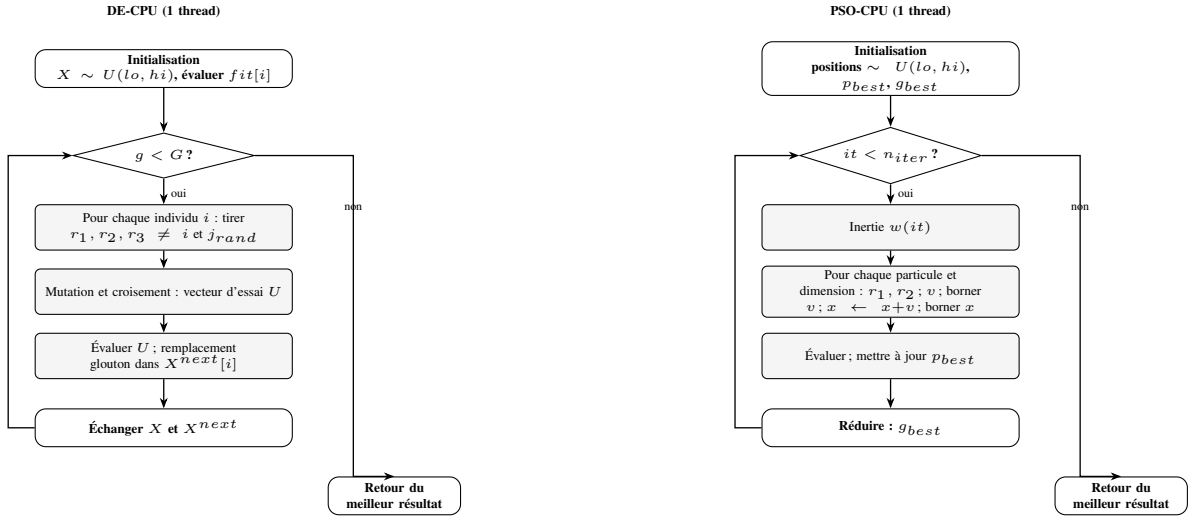
Nous décrivons deux versions CUDA de DE/rand/1/bin : la version de base de l'article [1] (Fig. 1 de l'article), où chaque opérateur est un kernel, puis une version améliorée qui fusionne les kernels. Les deux conservent exactement la sémantique du DE séquentiel de la section III-A.

1) DE CUDA de base (DE-GPU): La version de base suit la Fig. 1 de [1]. Chaque opérateur est un kernel séparé et les kernels communiquent par la mémoire globale (Fig. 2a). Le kernel I initialise la population et les états aléatoires et le kernel E l'évalue. Ensuite, chaque génération lance cinq kernels :

- **P** : un thread par individu tire r_1, r_2, r_3 (distincts et différents de i) et j_{rand}, rangés dans un `int4` ;
- **M** : mutation (1), un thread par élément de la population ;
- **C** : croisement (2), un thread par élément ;
- **E** : évaluation des vecteurs d'essai, avec *un bloc par individu* et D threads (arrondi à la puissance de deux supérieure), suivie d'une réduction en arbre en mémoire partagée ;
- **R** : remplacement glouton.

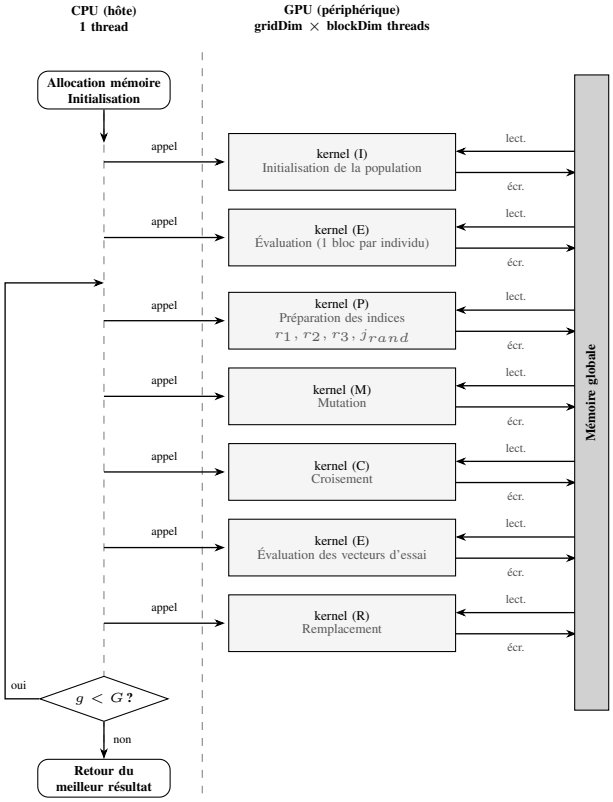
Les kernels I, M, C, R et P utilisent $TB = 256$ threads par bloc. Le tableau I résume le placement des threads de toutes les architectures.

Cette version est intuitive, mais elle a trois faiblesses : le trafic entre kernels, le coût de cinq lancements par génération,

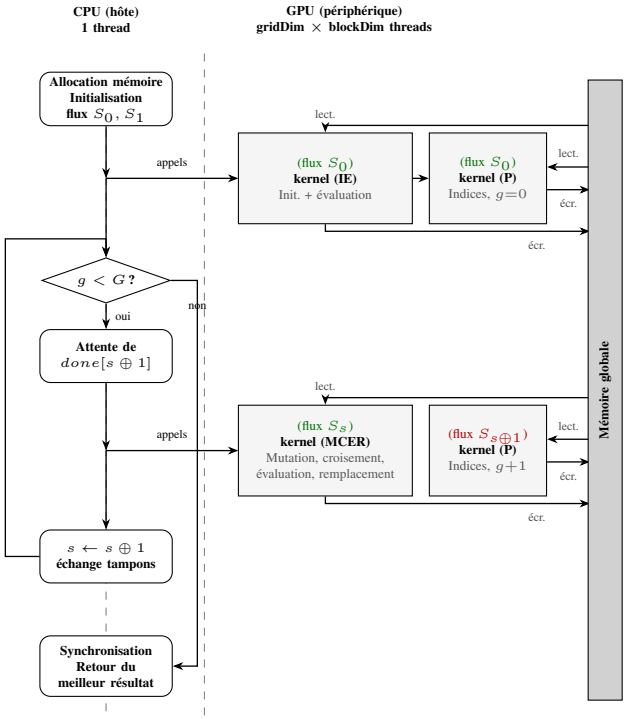


(a) DE-CPU (b) PSO-CPU

FIGURE 1. Organigrammes des versions séquentielles : (a) DE/rand/1/bin et (b) PSO. Tout s'exécute sur un seul thread de l'hôte ; les populations sont stockées par individu et les mises à jour sont synchrones (double tampon pour DE, meilleur global calculé une fois par génération pour PSO).



(a) DE-GPU



(b) DE-GPU-OPT

FIGURE 2. Organigrammes des versions CUDA de DE. (a) Version de base : un kernel par opérateur, l'encadré pointillé est répété à chaque génération. (b) Version améliorée : kernels IE, P et MCER sur deux flux S_s et $S_{s\oplus 1}$ ($s = g \bmod 2$) ; P(g+1) s'exécute pendant MCER(g) et l'événement $done[s \oplus 1]$ garantit l'ordre entre générations.

et une occupation des SM qui dépend de (P, D). Nous retrouvons ces effets dans les mesures (section V-C2).

*2) DE CUDA amélioré (*DE-GPU-OPT*):* La version améliorée conserve exactement la sémantique de DE/rand/1/bin, mais réorganise l'exécution autour de trois idées de [1]

TABLE I
PLACEMENT DES THREADS DANS LES KERNELS CUDA.

Architecture	Kernel	Placement
DE-GPU	I, M, C, R	1 thread par élément
	E	1 bloc par individu, réduction
	P	1 thread par individu
DE-GPU-OPT	IE, MCER	m individus par bloc, 1 thread par élément, mémoire partagée
	P	1 thread par individu, second flux
PSO-GPU	I, U	1 thread par élément, coordonnée-major
	E	1 thread par particule
	G	1 bloc, argmin en arbre

(Fig. 2b).

a) Fusion des kernels.: Les kernels I et E sont fusionnés en un kernel **IE**, et les kernels M, C, E et R en un kernel **MCER**. Le vecteur d'essai et les sommes partielles de l'évaluation restent en mémoire partagée pendant tout le kernel : la population n'est donc lue qu'une fois et écrite qu'une fois par génération, au lieu de plusieurs allers-retours en mémoire globale. Un bloc traite m individus avec un thread par coordonnée, soit mD threads, et trois tampons de mD doubles (`tr`, `sa`, `sb`) sont alloués en mémoire partagée dynamique. La population courante est lue dans `Xin` et la suivante est écrite dans un tampon distinct `Xout`, ce qui évite toute course de données (Listing 1).

```
const int i = blockIdx.x * m + mi; // individu
double v = Xin[id.x*D + j]
+ F * (Xin[id.y*D + j] - Xin[id.z*D + j]);
real u = curand_uniform_double(&s); // 1
aleatoire / element
trial = (u <= CR || j == id.w) ? v : xi;
tr[t] = trial; __syncthreads();
/* fit_term + reduction par segment de D threads
 */
bool acc = f < fcur; // remplacement glouton
Xout[e] = acc ? trial : xi;
if (j == 0) fitOut[i] = acc ? f : fcur;
```

Listing 1. Cœur du kernel MCER (extrait de `kernel.cu`).

b) Choix de la taille de bloc.: Il faut choisir le nombre m d'individus par bloc. L'article maximise m sous trois contraintes : les registres, la mémoire partagée et les limites du périphérique. Nous avons constaté que maximiser m peut laisser des SM inactifs sur les petites populations. Par exemple, pour $P = 50$ et $D = 100$, le plus grand m admissible est $m = 10$ (limite de 1 024 threads par bloc), donc seulement cinq blocs, soit cinq SM utilisés sur les quarante de la T4. Nous avons donc ajouté un critère de charge : le nombre de blocs est $\lceil P/m \rceil$, le nombre de warps par bloc est $\lceil mD/32 \rceil$ et la charge du SM le plus chargé vaut

$$L(m) = \left\lceil \frac{\lceil P/m \rceil}{N_{\mathrm{SM}}} \right\rceil \cdot \left\lceil \frac{mD}{32} \right\rceil . \tag{4}$$

La règle retient le m qui minimise $L(m)$ sous les contraintes matérielles ; les ex æquo sont départagés par la plus grande occupation (donnée par l'API `cudaOccupancyMaxActiveBlocksPerMultiprocessor`), puis par le plus grand m. Pour $P = 50$ et $D = 100$, on obtient $m = 2$ et 25 blocs (donc 25 SM utilisés). Le tableau II donne les valeurs de m obtenues sur la T4.

TABLE II
INDIVIDUS PAR BLOC m CHOISIS POUR LA T4 ($N_{\mathrm{SM}} = 40$).

D	$P = 50$	$P = 100$	$P = 500$
10	3	3	16
50	2	3	13
100	2	3	7

c) Deux flux.: Le kernel P (indices de mutation) ne dépend pas de la population de la génération en cours. On peut donc le lancer pour la génération suivante pendant que MCER travaille. Deux flux et deux tampons d'indices sont alternés à chaque génération, et deux événements CUDA garantissent que MCER ne démarre que lorsque la population de la génération précédente est complète (Listing 2). Les états aléatoires de P sont dédoublés (une famille par flux) pour éviter les courses.

```
for (long long g = 0; g < G; g++) {
    const int s = g & 1, c = s, nx = c ^ 1;
    if (g > 0) cudaStreamWaitEvent(S[s], done[
    s^1], 0);
    de_MCER<<<grid, T, smem, S[s]>>>(X[c], X[
    nx], fit[c],
    fit[nx], idx[s], stE, P, D, m, func, o, F,
     CR);
    cudaEventRecord(done[s], S[s]);
    if (g + 1 < G) // P de la generation
    suivante
    de_P<<<nblk(P), TB, 0, S[s^1]>>>(stP + (s
    ^1)*P,
    idx[s^1], P, D);
}
```

Listing 2. Boucle hôte de DE-GPU-OPT avec deux flux (extrait de `kernel.cu`).

d) Diagramme de séquence.: La figure 3 détaille l'enchaînement des appels de DE-GPU-OPT. L'hôte lance d'abord le kernel IE puis le kernel P de la génération 0 sur le flux S_0. Dans la boucle, il demande au flux S_s d'attendre l'événement $done[s \oplus 1]$ (fin de MCER$(g-1)$), lance MCER(g), enregistre l'événement $done[s]$, puis lance P$(g+1)$ sur l'autre flux $S_{s\oplus 1}$: MCER(g) et P$(g+1)$ s'exécutent donc en parallèle sur le GPU, et seul MCER dépend du résultat de la génération précédente. Après la dernière génération, l'hôte se synchronise et copie le meilleur individu.

IV. OPTIMISATION PAR ESSAIM PARTICULAIRE (PSO)

Le code CUDA fourni par l'enseignant implémente un PSO. Nous l'avons conservé comme second algorithme d'étude : il suit la même organisation que le DE (versions séquentielle et CUDA, mêmes fonctions de test, même budget, mêmes graines), ce qui permet de comparer deux algorithmes et deux approches de parallélisation.

A. Aperçu et version séquentielle (PSO-CPU)

Dans le PSO [3], chaque particule i a une position $\mathbf{x}_i$, une vitesse $\mathbf{v}_i$ et une meilleure position personnelle $\mathbf{p}_i$, et l'essaim a une meilleure position globale $\mathbf{g}$:

$$v_{i,j} \leftarrow w\, v_{i,j} + c_1 r_1 (p_{i,j} - x_{i,j}) + c_2 r_2 (g_j - x_{i,j}), \tag{5}$$

$$x_{i,j} \leftarrow x_{i,j} + v_{i,j}, \tag{6}$$

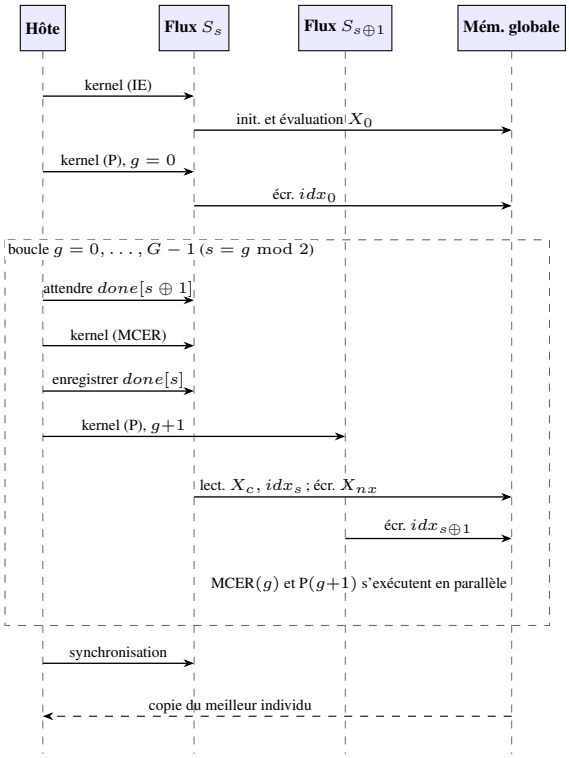


FIGURE 3. Diagramme de séquence de DE-GPU-OPT : IE, puis deux flux S_s et $S_{s\oplus1}$ avec $s = g \bmod 2$.

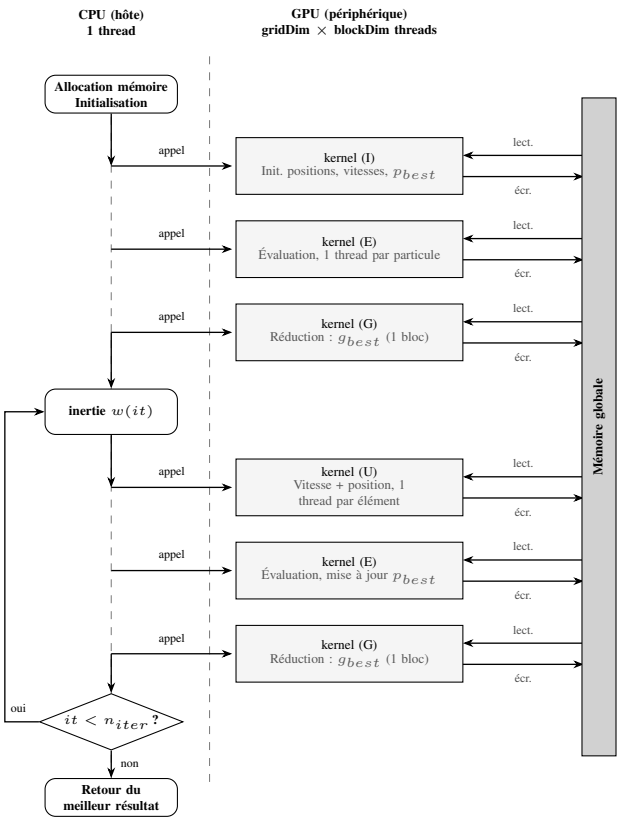


FIGURE 4. Organigramme du PSO CUDA (PSO-GPU) : kernels I, E et G avant la boucle, puis U, E et G à chaque itération. Le kernel E n'utilise qu'un thread par particule ; le kernel G est exécuté par un seul bloc.

avec $r_1, r_2 \sim U(0,1)$ tirés pour chaque particule et chaque dimension, $c_1 = c_2 = 2$, $|v| \leq 0{,}2(hi - lo)$, position bornée à $[lo, hi]$ (vitesse remise à zéro à la borne) et w décroissant linéairement de 0,9 à 0,4 [4]. Le meilleur global est mis à jour une seule fois par itération (version synchrone), après l'évaluation de tout l'essaim. Comme pour DE, le budget est $\mathrm{MaxFEs} = 10^4 D$: le nombre d'itérations est $n_{iter} = \lfloor \mathrm{MaxFEs}/P \rfloor - 1$, soit $P(1 + n_{iter})$ évaluations. La figure 1b donne l'organigramme de PSO-CPU.

B. Version massivement parallèle (PSO-GPU)

Le PSO-GPU (Fig. 4) utilise quatre kernels : I (initialisation), E (évaluation et mise à jour de $\mathbf{p}_i$), G (réduction en arbre dans un seul bloc pour $\mathbf{g}$) et U (mise à jour de la vitesse et de la position, un thread par élément). L'essaim est stocké par coordonnée ($\mathrm{pos}[d \cdot P + p]$) pour que des threads consécutifs lisent des adresses consécutives. En revanche, le kernel E n'utilise qu'*un thread par particule* qui boucle sur les D coordonnées : seuls P threads travaillent, ce qui limite l'accélération (section V-C2). Contrairement à DE, la mise à jour d'une particule dépend du meilleur global : le kernel G sépare donc deux phases de calcul et il n'y a pas de recouvrement possible par des flux.

V. RÉSULTATS EXPÉRIMENTAUX

A. Configuration

Les expériences ont été réalisées dans un notebook Kaggle avec un GPU NVIDIA Tesla T4 (40 SM, 15 Go, capacité 7.5, pilote 580.178.04) et CUDA 12.8. Le CPU hôte est un Xeon de la machine virtuelle Kaggle, utilisé avec un seul thread. Le code est compilé avec `nvcc -O3 -std=c++17 -arch=sm_75`. Les populations et les fitness sont en double précision, ce qui pénalise la T4 dont le débit en double précision est faible. Le temps mesuré est celui de l'optimisation : les allocations et la création du contexte CUDA sont exclues (un lancement de chauffe est fait avant). Les cinq architectures utilisent les mêmes paramètres ($F = 0{,}5$, $CR = 0{,}3$ pour DE) et le run k utilise la graine $1234 + k$ pour tous les algorithmes. À chaque run, la meilleure fitness du GPU est recalculée sur l'hôte et comparée à celle du GPU (tolérance relative 10^{-6}). Le nombre d'évaluations consommé vaut exactement $P(1 + G) = \mathrm{MaxFEs}$ pour les 1 800 exécutions. En outre, la meilleure fitness rapportée par chaque exécution GPU est recalculée sur l'hôte avec la même fonction : aucun écart (tolérance relative 10^{-6}) n'a été signalé.

B. Fonctions de test

Nous utilisons les quatre fonctions décalées de CEC2005 [5], avec $\mathbf{z} = \mathbf{x} - \mathbf{o}$ et $f(\mathbf{o}) = f^*$ (Tableau III). Pour Rosenbrock, $\mathbf{z} = \mathbf{x} - \mathbf{o} + 1$. Les vecteurs de décalage de CEC2005 sont intégrés dans le code et identiques pour toutes les architectures. Un auto-test (`./cudade -verify`) vérifie

TABLE III
FONCTIONS DE TEST DÉCALÉES.

	Expression	Domaine	f^*
F_1 Sphere	$\sum_i z_i^2$	$[-100, 100]^D$	-450
F_2 Rosenbrock	$\sum_{i=1}^{D-1} \left[100(z_i^2 - z_{i+1})^2 + (z_i - 1)^2\right]$	$[-100, 100]^D$	390
F_3 Griewank	$\sum_i \frac{z_i^2}{4000} - \prod_i \cos \frac{z_i}{\sqrt{i}} + 1$	$[-600, 600]^D$	-180
F_4 Rastrigin	$\sum_i \left[z_i^2 - 10\cos(2\pi z_i) + 10\right]$	$[-5, 5]^D$	-330

TABLE IV
TEMPS MOYEN D'UNE EXÉCUTION (MS), MOYENNÉ SUR LES QUATRE FONCTIONS (40 EXÉCUTIONS PAR CASE).

D	P	PSO-CPU	PSO-GPU	DE-CPU	DE-GPU	DE-GPU-OPT
10	50	37.8	38.2	28.0	31.8	11.6
10	100	38.2	21.0	29.3	14.7	5.6
10	500	39.8	6.3	33.5	3.8	1.3
50	50	886.9	499.9	632.9	150.0	60.0
50	100	890.8	316.9	636.4	77.5	32.0
50	500	940.5	121.1	691.2	28.9	20.3
100	50	3553.8	1813.9	2515.9	322.0	154.0
100	100	3584.6	1226.9	2532.7	186.0	102.0
100	500	3740.9	485.0	2784.6	113.9	84.4

$f(\mathbf{o}) = f^*$ pour les quatre fonctions et $D \in \{10, 50, 100\}$; il est réussi dans les 12 cas.

Sphere est unimodale et sert à valider les implémentations. Rosenbrock a une vallée étroite et courbe, difficile à suivre. Griewank est multimodale mais le produit de cosinus s'atténue loin de l'optimum. Rastrigin est fortement multimodale avec de nombreux optima locaux réguliers.

C. Résultats

1) Protocole expérimental: Nous suivons le protocole du projet : $D \in \{10, 50, 100\}$, $P \in \{50, 100, 500\}$, MaxFEs $= 10^4 D$ et 10 exécutions indépendantes par cas, soit $4 \times 3 \times 3 \times 10 = 360$ exécutions par architecture. Pour chaque exécution, nous relevons l'erreur finale EFV $= f(\mathbf{x}_{best}) - f^*$, le temps et le succès (EFV $< 10^{-8}$). L'article utilise aussi $P = 1000$ et 25 exécutions ; nous n'avons pas lancé $P = 1000$, qui ne figure pas dans les consignes du projet, et nous avons fait 10 exécutions.

2) Temps de calcul et accélérations: Le Tableau IV donne le temps moyen d'une exécution (moyenne sur les quatre fonctions) et la Fig. 5 détaille les temps par fonction. Le temps CPU ne dépend presque pas de P : le nombre d'évaluations est fixe. Le temps GPU diminue quand P augmente, car il y a moins de générations et donc moins de lancements de kernels.

Le Tableau V (et la Fig. 6) donne les accélérations. DE-GPU-OPT est de 2,4× ($D = 10$, $P = 50$) à 33,2× ($D = 50$, $P = 500$) plus rapide que DE-CPU. Par rapport à DE-GPU, le gain va de 1,35× à 2,90× et il est le plus fort sur les petites populations, où le coût de lancement domine. DE-GPU est *plus lent* que le CPU sur le plus petit problème (0,91× pour $D = 10$, $P = 50$) : cinq lancements par génération ne sont pas compensés par le calcul. Le PSO-GPU atteint au plus 9,4×.

Pour comprendre ces valeurs, nous avons calculé le coût d'une génération (Tableau VI). Pour $D = 10$ et $P = 50$, DE-GPU coûte 15,9 μs par génération, soit environ 3 μs par kernel pour cinq kernels, ce qui correspond au coût d'un lancement : on est limité par la latence de lancement, non par le calcul. DE-GPU-OPT coûte 5,8 μs, soit deux kernels dont l'un est recouvert. Quand P et D augmentent, chaque génération contient plus de calcul et le coût CPU croît fortement, alors que le coût GPU croît peu : c'est là que l'accélération est maximale.

TABLE V
ACCÉLÉRATIONS (RAPPORT DES TEMPS MOYENS, MOYENNE SUR LES QUATRE FONCTIONS). UNE VALEUR > 1 SIGNIFIE QUE LE PREMIER ALGORITHME EST PLUS RAPIDE.

D	P	PSO-GPU /PSO-CPU	DE-GPU /DE-CPU	DE-GPU-OPT/DE-CPU	DE-GPU-OPT/DE-GPU
10	50	0.99	0.91	2.41	2.75
10	100	1.84	1.97	5.25	2.64
10	500	6.87	8.50	24.77	2.90
50	50	1.96	4.18	10.52	2.51
50	100	3.31	8.12	19.45	2.43
50	500	9.41	23.28	33.16	1.43
100	50	2.26	7.73	16.16	2.12
100	100	3.53	13.43	24.53	1.85
100	500	9.40	23.59	31.86	1.35

TABLE VI
COÛT MOYEN D'UNE GÉNÉRATION (μS).

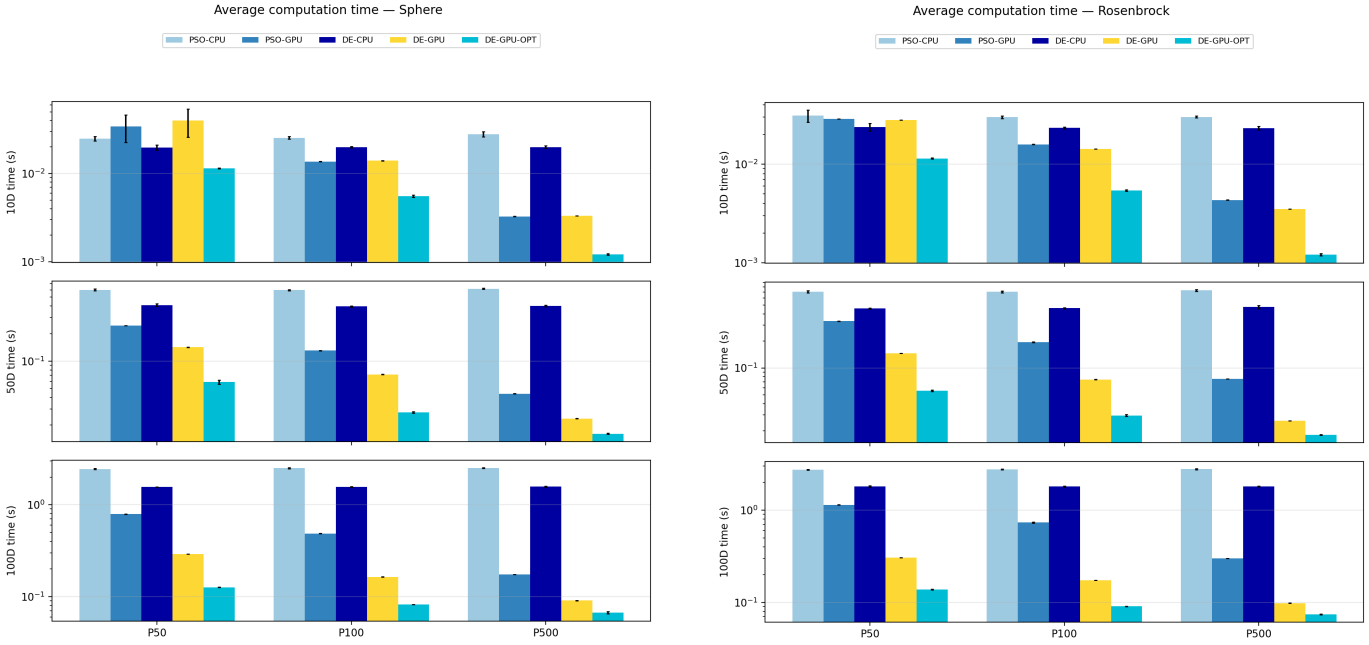
D	P	PSO-CPU	PSO-GPU	DE-CPU	DE-GPU	DE-GPU-OPT
10	50	18.9	19.1	14.0	15.9	5.8
10	100	38.2	21.0	29.3	14.7	5.6
10	500	200.1	31.5	168.4	19.3	6.6
50	50	88.7	50.0	63.3	15.0	6.0
50	100	178.2	63.4	127.3	15.5	6.4
50	500	941.4	121.2	691.9	28.9	20.3
100	50	177.7	90.7	125.8	16.1	7.7
100	100	358.5	122.7	253.3	18.6	10.2
100	500	1871.4	242.6	1393.0	57.0	42.2

TABLE VII
PERFORMANCE DE DE-GPU-OPT : MOYENNE (ÉCART-TYPE) DE LA MEILLEURE EFV, DU TEMPS (S) ET DU TAUX DE SUCCÈS SUR 10 EXÉCUTIONS, POUR TROIS TAILLES DE POPULATION, PROBLÈMES EN 10D.

		P50	**P100**	**P500**
F_1	Best EFV	0.000 (0.000)	0.000 (0.000)	1.23×10^{-5} (2.67×10^{-6})
	Temps (s)	0.012 (0.000)	0.006 (0.000)	0.001 (0.000)
	SR	1.00	1.00	0.00
F_2	Best EFV	0.208 (0.214)	4.407 (2.105)	71.35 (18.99)
	Temps (s)	0.011 (0.000)	0.005 (0.000)	0.001 (0.000)
	SR	0.00	0.00	0.00
F_3	Best EFV	0.000 (0.000)	0.000 (0.000)	0.143 (0.023)
	Temps (s)	0.012 (0.001)	0.006 (0.000)	0.002 (0.000)
	SR	1.00	1.00	0.00
F_4	Best EFV	0.000 (0.000)	0.000 (0.000)	5.436 (0.976)
	Temps (s)	0.012 (0.000)	0.006 (0.000)	0.001 (0.000)
	SR	1.00	1.00	0.00

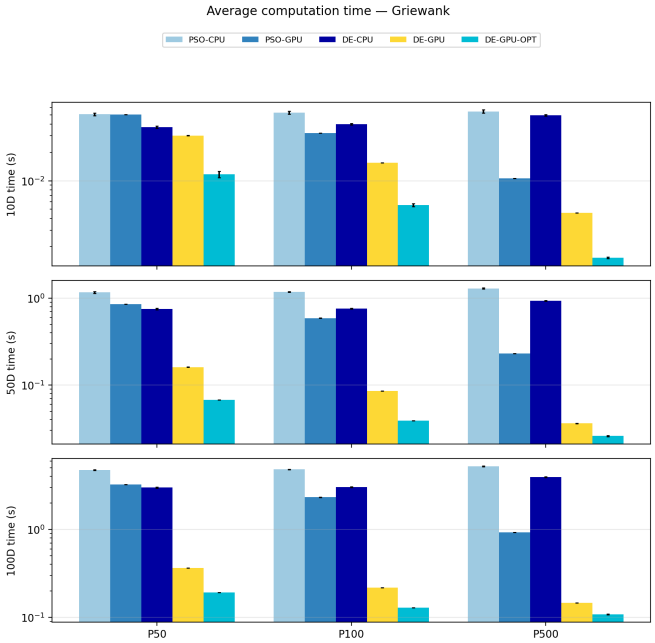
3) Qualité des solutions: Les Tableaux VII, VIII et IX présentent DE-GPU-OPT dans le même format que les Tableaux 1 à 3 de [1] : moyenne et écart-type (entre parenthèses) de l'erreur finale, du temps et du taux de succès (SR). Les valeurs inférieures à 10^{-8} sont affichées `0.000`.

a) Effet de la taille de population.: À budget fixe, $P = 500$ ne donne que 199 ($D = 10$) à 1 999 ($D = 100$)

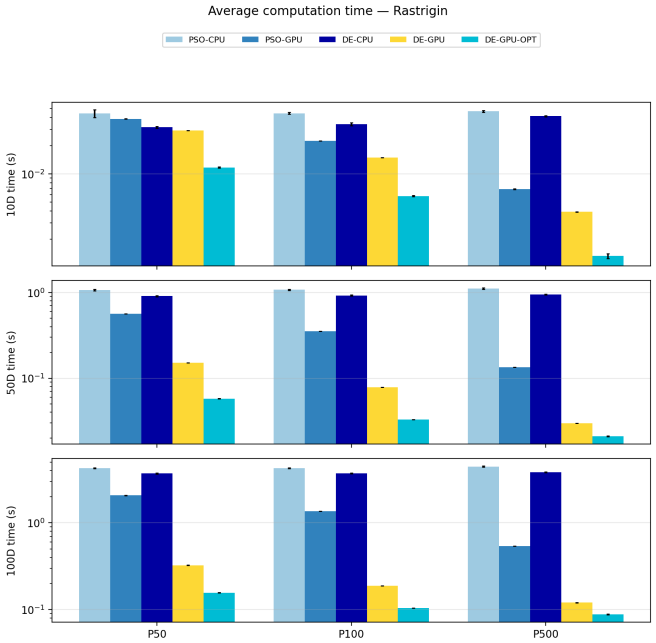


(a) F_1 Sphere

(b) F_2 Rosenbrock



(c) F_3 Griewank



(d) F_4 Rastrigin

FIGURE 5. Temps de calcul moyen (échelle logarithmique) des cinq architectures, pour chaque fonction, $D \in \{10, 50, 100\}$ et $P \in \{50, 100, 500\}$ (10 exécutions ; barres : écart-type).

générations et aucun succès (SR$= 0$) sur aucune des quatre fonctions, alors que Sphere et Griewank sont résolues (SR$= 1$) pour $P = 50$ et $P = 100$ à toutes les dimensions ; Rastrigin l'est en 10D. Une grande population n'améliore donc pas la qualité, ce qui rejoint l'observation de [1] selon laquelle elle réduit seulement le temps. Rosenbrock n'est jamais résolue (SR$= 0$).

b) Comparaison avec l'article de référence.: Le Tableau X compare l'EFV moyenne de DE-GPU-OPT à celle de cudaDE$_i$ dans [1]. Les deux budgets nominaux sont identiques ($10^4 D$ évaluations). Nos résultats concordent pour Sphere et Griewank avec $P = 50$ et $P = 100$ (erreur nulle ou quasi nulle), et pour Rosenbrock ils sont du même ordre (meilleurs en 50D avec $P = 50$: 51,7 contre 213,5 ; moins bons en 100D : 97,4 contre 15,7). En revanche, ils diffèrent : (i) pour $P = 500$, où l'article atteint des erreurs nulles sur Sphere (toutes dimensions), sur Griewank et Rastrigin en 10D, et 113 sur Rosenbrock 100D, alors que nous obtenons de $1{,}2 \times 10^{-5}$

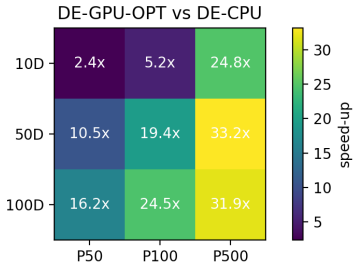


FIGURE 6. Accélération de DE-GPU-OPT par rapport à DE-CPU (moyenne sur les quatre fonctions).

TABLE VIII
MÊME FORMAT QUE LE TABLEAU VII, PROBLÈMES EN 50D.

		P50	**P100**	**P500**
F_1	Best EFV	0.000 (0.000)	0.000 (0.000)	5.70×10^{-4} (6.19×10^{-5})
	Temps (s)	0.059 (0.003)	0.028 (0.001)	0.016 (0.000)
	SR	1.00	1.00	0.00
F_2	Best EFV	51.72 (22.70)	60.62 (27.78)	1976.0 (264.5)
	Temps (s)	0.056 (0.001)	0.030 (0.001)	0.018 (0.000)
	SR	0.00	0.00	0.00
F_3	Best EFV	0.000 (0.000)	0.000 (0.000)	1.02×10^{-3} (1.59×10^{-4})
	Temps (s)	0.067 (0.000)	0.039 (0.000)	0.026 (0.000)
	SR	1.00	1.00	0.00
F_4	Best EFV	142.1 (8.804)	167.0 (8.782)	237.1 (10.61)
	Temps (s)	0.058 (0.000)	0.033 (0.000)	0.021 (0.000)
	SR	0.00	0.00	0.00

TABLE IX
MÊME FORMAT QUE LE TABLEAU VII, PROBLÈMES EN 100D.

		P50	**P100**	**P500**
F_1	Best EFV	0.000 (0.000)	0.000 (0.000)	0.344 (0.026)
	Temps (s)	0.126 (0.000)	0.082 (0.000)	0.067 (0.002)
	SR	1.00	1.00	0.00
F_2	Best EFV	97.38 (21.82)	109.5 (28.98)	556100.0 (117700.0)
	Temps (s)	0.138 (0.001)	0.091 (0.000)	0.074 (0.000)
	SR	0.00	0.00	0.00
F_3	Best EFV	0.000 (0.000)	0.000 (0.000)	0.206 (0.015)
	Temps (s)	0.192 (0.000)	0.129 (0.000)	0.108 (0.001)
	SR	1.00	1.00	0.00
F_4	Best EFV	543.3 (16.76)	583.4 (15.49)	701.6 (19.12)
	Temps (s)	0.157 (0.000)	0.105 (0.000)	0.089 (0.001)
	SR	0.00	0.00	0.00

(Sphere 10D) à 0,34 (Sphere 100D), 0,14 (Griewank 10D), 5,4 (Rastrigin 10D) et $5{,}6\times10^5$ (Rosenbrock 100D) ; (ii) pour Rastrigin en 50D et 100D, où nous obtenons 142 et 543 avec $P = 50$ contre 18,3 et 63,2 dans l'article. Nous n'avons pas pu identifier la cause de ces écarts. Les temps rapportés dans [1] sont de l'ordre de 100 fois supérieurs aux nôtres ; le matériel de 2012 (GTX 560, Xeon 2,33 GHz) n'explique sans doute qu'une partie de cet écart, et nous ne pouvons pas affirmer que l'article utilise le même nombre d'évaluations que celui annoncé. Nous avons aussi réalisé 10 exécutions au lieu de 25, ce qui change la précision statistique (SR par pas de 0,1 au lieu de 0,04) mais pas l'espérance de l'EFV.

4) Comparaison DE et PSO: Les tableaux XI, XII et XIII donnent la moyenne et l'écart-type de l'EFV des cinq architectures pour les 36 cas (la Fig. 7 montre l'évolution avec P). DE est nettement meilleur que PSO sur Sphere, Griewank et Rosenbrock à 50D et 100D : par exemple, sur Sphere 100D avec $P = 50$, DE atteint la précision machine alors que PSO reste à 4×10^3. En revanche, *PSO est meilleur que DE sur Rastrigin à 50D et 100D* (environ 82 contre 143 en 50D, 279 contre 546 en 100D avec $P = 50$). Pour un même algorithme, les versions CPU et GPU donnent des EFV du même ordre : le portage ne modifie pas le comportement de l'algorithme. Cette comparaison reste partielle, car les paramètres du PSO (inertie de 0,9 à 0,4, $c_1 = c_2 = 2$) ne sont pas réglés et peuvent être instables en grande dimension (voir section V-C6).

TABLE X
EFV MOYENNE : CUDADE$_i$ DE L'ARTICLE [1] (25 EXÉCUTIONS, GTX 560) ET DE-GPU-OPT (10 EXÉCUTIONS, T4). VALEURS INFÉRIEURES À 10^{-8} AFFICHÉES `0.000`.

		P50		P100		P500	
	D	Art.	Nous	Art.	Nous	Art.	Nous
F_1	10	0.000	0.000	0.000	0.000	0.000	1.23×10^{-5}
	50	0.000	0.000	0.000	0.000	0.000	5.70×10^{-4}
	100	0.000	0.000	0.000	0.000	0.000	0.344
F_2	10	0.388	0.208	0.044	4.407	1.306	71.35
	50	213.5	51.72	12.24	60.62	46.97	1976.0
	100	15.72	97.38	42.29	109.5	113.3	556100.0
F_3	10	0.000	0.000	0.000	0.000	0.000	0.143
	50	0.001	0.000	0.000	0.000	0.000	1.02×10^{-3}
	100	0.002	0.000	0.005	0.000	0.000	0.206
F_4	10	0.000	0.000	0.000	0.000	0.000	5.436
	50	18.29	142.1	2.936	167.0	28.35	237.1
	100	63.17	543.3	48.33	583.4	40.89	701.6

5) Tests statistiques: Nous comparons les distributions d'EFV (10 valeurs par algorithme et par cas) pour chacun des 36 cas (fonction, dimension, population). Un test de Kruskal-Wallis porte sur les cinq algorithmes ; les comparaisons deux à deux utilisent le test de Mann-Whitney (échantillons indépendants, bilatéral, seuil de 5 %), avec la correction de Holm sur les 36 cas de chaque comparaison. Nous avons choisi un test non apparié, car les exécutions d'algorithmes différents partagent la graine mais pas le générateur aléatoire (Mersenne Twister sur CPU, Philox sur GPU) : le k-ième run d'un algorithme n'est pas « le même » que celui d'un autre. Les cas où les deux échantillons sont identiques (toutes les erreurs sont nulles) sont comptés comme non significatifs. Le tableau XIV résume les résultats.

Les trois versions de DE sont statistiquement indiscernables : aucun cas significatif entre DE-CPU, DE-GPU et DE-GPU-OPT après correction, et un seul avant correction (Rosenbrock 10D, $P = 50$, $p = 0{,}021$, que la correction de Holm ne conserve pas, $p_{Holm} = 0{,}76$; avec 36 tests, un faux positif est attendu). La parallélisation et l'optimisation ne modifient donc pas la qualité des solutions. En revanche, DE et PSO diffèrent dans 30 à 32 cas sur 36. Les cinq cas non significatifs entre DE et PSO sont Sphere 10D ($P = 50$ et $P = 100$, où les deux atteignent l'erreur nulle), Rosenbrock

TABLE XI
EFV : MOYENNE (ÉCART-TYPE) SUR 10 EXÉCUTIONS POUR LES CINQ ARCHITECTURES, $D = 10$. VALEURS INFÉRIEURES À 10^{-8} AFFICHÉES 0.

Fonction	P	PSO-CPU	PSO-GPU	DE-CPU	DE-GPU	DE-GPU-OPT
Sphere	50	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
	100	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
	500	5.3e-7 (6.7e-7)	4.0e-7 (4.3e-7)	1.4e-5 (4.2e-6)	1.2e-5 (3.9e-6)	1.2e-5 (2.7e-6)
Rosenbrock	50	15.6 (29.4)	49.6 (106)	0.604 (0.846)	2.09 (1.96)	0.208 (0.214)
	100	23.1 (36.8)	7.9e4 (2.5e5)	4.08 (1.9)	2.99 (2.7)	4.41 (2.1)
	500	52.7 (70.6)	21.8 (25.7)	66.2 (10.3)	59.3 (15.5)	71.3 (19)
Griewank	50	0.117 (0.106)	0.164 (0.0786)	0 (0)	0 (0)	0 (0)
	100	0.124 (0.082)	0.111 (0.0608)	0 (0)	0 (0)	0 (0)
	500	0.194 (0.0686)	0.154 (0.0707)	0.121 (0.0351)	0.145 (0.0243)	0.143 (0.023)
Rastrigin	50	1.79 (1.13)	2.09 (1.09)	0 (0)	0 (0)	0 (0)
	100	2.49 (1.5)	2.09 (1.09)	0 (0)	0 (0)	0 (0)
	500	2.97 (1.17)	3.3 (1.47)	5.96 (1.04)	5.85 (1)	5.44 (0.976)

TABLE XII
EFV : MOYENNE (ÉCART-TYPE) SUR 10 EXÉCUTIONS POUR LES CINQ ARCHITECTURES, $D = 50$. VALEURS INFÉRIEURES À 10^{-8} AFFICHÉES 0.

Fonction	P	PSO-CPU	PSO-GPU	DE-CPU	DE-GPU	DE-GPU-OPT
Sphere	50	548 (280)	1.1e+03 (1.5e+03)	0 (0)	0 (0)	0 (0)
	100	399 (393)	700 (501)	0 (0)	0 (0)	0 (0)
	500	543 (885)	79.5 (123)	6.1e-4 (8.1e-5)	6.1e-4 (6.6e-5)	5.7e-4 (6.2e-5)
Rosenbrock	50	4.7e8 (1.4e9)	1.5e7 (1.5e7)	50.5 (22.9)	68.9 (32.2)	51.7 (22.7)
	100	5.6e6 (8.8e6)	9.3e6 (1.4e7)	77.4 (29.7)	55 (24.7)	60.6 (27.8)
	500	3.6e5 (7.7e5)	3.8e6 (5.9e6)	1.96e+03 (520)	2.16e+03 (577)	1.98e+03 (265)
Griewank	50	12.4 (8.73)	16.9 (19.3)	0 (0)	0 (0)	0 (0)
	100	13.5 (11.4)	9.94 (10.6)	0 (0)	0 (0)	0 (0)
	500	8.62 (13.9)	7.15 (7.45)	1.1e-3 (1.6e-4)	9.5e-4 (1.8e-4)	1.0e-3 (1.6e-4)
Rastrigin	50	82.3 (15.9)	88.7 (18.1)	143 (12.6)	144 (7.89)	142 (8.8)
	100	80.4 (13.6)	88.8 (15.6)	172 (9.97)	173 (6.91)	167 (8.78)
	500	89.4 (19.6)	83.3 (16.8)	241 (6.82)	240 (7.72)	237 (10.6)

TABLE XIII
EFV : MOYENNE (ÉCART-TYPE) SUR 10 EXÉCUTIONS POUR LES CINQ ARCHITECTURES, $D = 100$. VALEURS INFÉRIEURES À 10^{-8} AFFICHÉES 0.

Fonction	P	PSO-CPU	PSO-GPU	DE-CPU	DE-GPU	DE-GPU-OPT
Sphere	50	4.1e+03 (1.09e+03)	4.18e+03 (1.69e+03)	0 (0)	0 (0)	0 (0)
	100	2.52e+03 (932)	3.51e+03 (3.12e+03)	0 (0)	0 (0)	0 (0)
	500	814 (428)	973 (645)	0.338 (0.0266)	0.341 (0.0173)	0.344 (0.0261)
Rosenbrock	50	3.4e8 (4.1e8)	1.9e8 (1.3e8)	112 (28)	101 (26.8)	97.4 (21.8)
	100	2.4e8 (4.0e8)	8.0e7 (5.6e7)	97.1 (19)	96.9 (18.8)	110 (29)
	500	1.6e7 (1.9e7)	2.3e7 (3.0e7)	5.9e5 (9.7e4)	5.8e5 (1.7e5)	5.6e5 (1.2e5)
Griewank	50	39.4 (14.5)	43.4 (44.1)	0 (0)	0 (0)	0 (0)
	100	24.6 (10.6)	24.9 (10.3)	0 (0)	0 (0)	0 (0)
	500	34.4 (58.4)	13.8 (9.89)	0.205 (0.0111)	0.202 (0.0135)	0.206 (0.0146)
Rastrigin	50	279 (46.4)	276 (41.2)	546 (14.4)	543 (13.9)	543 (16.8)
	100	227 (38.3)	244 (37.2)	588 (15.8)	585 (24.8)	583 (15.5)
	500	269 (19.8)	245 (27.8)	710 (17.6)	710 (14.6)	702 (19.1)

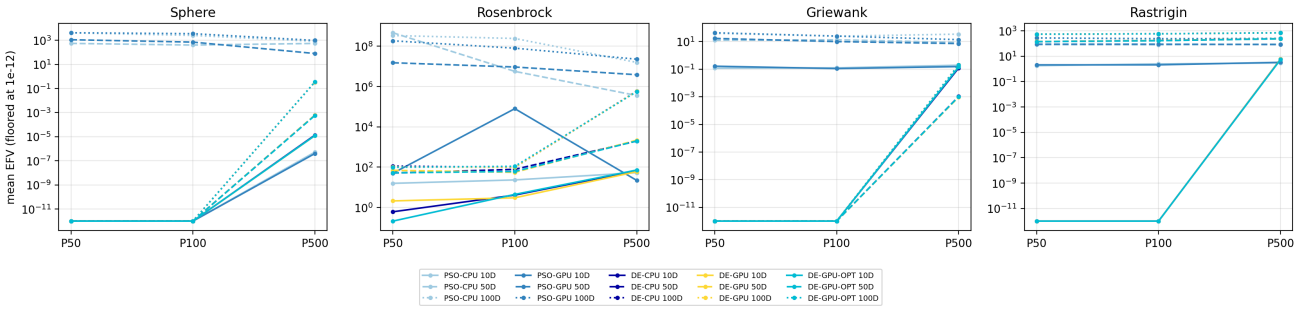


FIGURE 7. EFV moyenne (échelle logarithmique, plancher à 10^{-12}) en fonction de P, pour chaque fonction, algorithme et dimension (trait plein : 10D, tirets : 50D, pointillés : 100D).

10D ($P = 100$) et 50D ($P = 500$), et Griewank 10D ($P = 500$ pour DE-GPU-OPT seulement). Le test de Kruskal-Wallis est significatif dans 31 cas ; il ne l'est pas dans ces mêmes cinq cas. Un test de Wilcoxon apparié sur les graines, que nous avons aussi calculé, donne des comptes très proches (1, 2, 31 et 30 cas pour les quatre comparaisons ci-dessus) ; nous ne le retenons pas, pour la raison d'appariement donnée plus haut.

TABLE XIV
NOMBRE DE CAS SUR 36 OÙ LES DISTRIBUTIONS D'EFV DIFFÈRENT SIGNIFICATIVEMENT ($p < 0{,}05$), SANS ET AVEC CORRECTION DE HOLM (MANN-WHITNEY ; KRUSKAL-WALLIS SANS CORRECTION).

Comparaison	$p < 0{,}05$	Holm
DE-GPU-OPT vs DE-CPU	0/36	0/36
DE-GPU-OPT vs DE-GPU	1/36	0/36
DE-GPU vs DE-CPU	0/36	0/36
DE-GPU-OPT vs PSO-GPU	31/36	30/36
DE-CPU vs PSO-CPU	32/36	30/36
Kruskal-Wallis (cinq algorithmes)	31/36	—

6) Étude complémentaire : réglage des paramètres: Le projet encourage à améliorer la qualité des résultats. Nous avons donc étudié trois leviers, avec 10 exécutions par cas, sur la même machine (T4) et avec le même programme : le taux de croisement CR, le budget d'évaluations et les paramètres du PSO. Le tableau principal garde les paramètres de [1] ($F = 0{,}5$, $CR = 0{,}3$) ; cette section ne sert qu'à comprendre leur influence. Tous les essais utilisent $P = 50$ sauf mention contraire.

a) Taux de croisement.: Le tableau XV donne l'EFV moyenne de DE-GPU-OPT (et le taux de succès entre parenthèses) pour trois valeurs de CR. Sphere et Griewank sont résolues quelle que soit la valeur de CR (sauf Griewank avec $CR = 0{,}9$). Rastrigin est la fonction la plus sensible : avec $CR = 0{,}1$, elle est résolue en 50D (erreur de l'ordre de 10^{-13}, SR$= 1$) et passe de 543 à 151 en 100D ; avec $CR = 0{,}9$, elle passe de 142 à 43 en 50D mais se dégrade en 10D (SR de 1,0 à 0,7). Rastrigin est séparable : un faible CR ne modifie que peu de coordonnées à la fois, ce qui permet d'optimiser chaque coordonnée presque indépendamment. À l'inverse, Rosenbrock, dont les variables sont couplées, préfère $CR = 0{,}3$: $CR = 0{,}1$ et $CR = 0{,}9$ la dégradent, et très fortement $CR = 0{,}9$ en grande dimension (670 en 50D et 1 199 en 100D, contre 52 et 97). Aucune valeur de CR n'est donc la meilleure pour toutes les fonctions. Les versions DE-CPU et DE-GPU-OPT donnent les mêmes ordres de grandeur ; avec $CR = 0{,}9$, les moyennes de Rosenbrock sont dominées par quelques runs très mauvais (écart-type 2 148 pour une moyenne de 1 199 avec DE-GPU-OPT en 100D).

b) Budget multiplié par 10.: Avec $\mathrm{MaxFEs} = 10^5 D$ (tableau XVI), Sphere et Griewank sont résolues (SR$= 1$) en 10D et en 50D pour $P = 50$ comme pour $P = 500$, et Rastrigin l'est en 10D pour les deux tailles de population : un budget dix fois plus grand supprime l'écart de $P = 500$ en 10D que nous avions observé avec le budget nominal. Rosenbrock s'en approche pour $P = 50$ ($6{,}2 \times 10^{-3}$ avec SR$= 0{,}8$ en 10D ; 0,08 avec SR$= 0{,}3$ en 50D) mais pas pour $P = 500$ (0,14 en 10D, 38,7 en 50D). En revanche, Rastrigin en 50D reste à 62 ($P = 50$) et 141 ($P = 500$) alors que l'article rapporte 18,3 pour $P = 50$: un budget plus grand ne suffit donc pas à expliquer l'écart avec l'article.

c) PSO avec facteur de constriction.: Le tableau XVII compare nos paramètres initiaux (w de 0,9 à 0,4, $c_1 = c_2 = 2$) à la constriction de Clerc ($w = 0{,}7298$, $c_1 = c_2 = 1{,}49618$).

TABLE XV
EFFET DU TAUX DE CROISEMENT SUR DE-GPU-OPT ($P = 50$) : EFV MOYENNE (TAUX DE SUCCÈS).

Fonction	D	$CR = 0{,}1$	$CR = 0{,}3$	$CR = 0{,}9$
Sphere	10	0 (1.0)	0 (1.0)	0 (1.0)
	50	0 (1.0)	0 (1.0)	0 (1.0)
	100	0 (1.0)	0 (1.0)	0 (1.0)
Rosenbrock	10	1.7 (0.0)	0.208 (0.0)	2.32 (0.1)
	50	62.1 (0.0)	51.7 (0.0)	670 (0.0)
	100	112 (0.0)	97.4 (0.0)	1199 (0.0)
Griewank	10	0 (1.0)	0 (1.0)	0.0192 (0.5)
	50	0 (1.0)	0 (1.0)	1.7×10^{-3} (0.8)
	100	0 (1.0)	0 (1.0)	0.0277 (0.7)
Rastrigin	10	0 (1.0)	0 (1.0)	0.298 (0.7)
	50	0 (1.0)	142 (0.0)	42.6 (0.0)
	100	151 (0.0)	543 (0.0)	148 (0.0)

TABLE XVI
EFFET DU BUDGET SUR DE-GPU-OPT : EFV MOYENNE AVEC $10^4 D$ ÉVALUATIONS, PUIS AVEC $10^5 D$ ÉVALUATIONS (TAUX DE SUCCÈS). PREMIÈRE COLONNE : $D\,/\,P$.

D/P	Fonction	$10^4 D$	$10^5 D$
10 / 50	Sphere	0	0 (1.0)
	Rosenbrock	0.208	6.2×10^{-3} (0.8)
	Griewank	0	0 (1.0)
	Rastrigin	0	0 (1.0)
10 / 500	Sphere	1.2×10^{-5}	0 (1.0)
	Rosenbrock	71.3	0.145 (0.0)
	Griewank	0.143	0 (1.0)
	Rastrigin	5.44	0 (1.0)
50 / 50	Sphere	0	0 (1.0)
	Rosenbrock	51.7	0.0817 (0.3)
	Griewank	0	0 (1.0)
	Rastrigin	142	61.6 (0.0)
50 / 500	Sphere	5.7×10^{-4}	0 (1.0)
	Rosenbrock	1976	38.7 (0.0)
	Griewank	1.0×10^{-3}	0 (1.0)
	Rastrigin	237	141 (0.0)

La constriction améliore nettement Sphere, Griewank et Rosenbrock en grande dimension : par exemple Sphere 100D passe de 4 096 à 1 575 avec PSO-CPU, et Rosenbrock 50D de $4{,}7{\times}10^8$ à 42,5. Elle dégrade en revanche Rastrigin à toutes les dimensions (de 82 à 253 en 50D, de 279 à 630 en 100D avec PSO-CPU), et même en 10D (de 1,8 à 8,6). Les moyennes de Rosenbrock sont très dispersées (écart-type supérieur à la moyenne pour PSO-GPU en 50D, $5{,}7 \times 10^6$ en moyenne), si bien que les écarts entre PSO-CPU et PSO-GPU sur cette fonction ne sont pas interprétables avec 10 exécutions.

d) Bilan.: Le réglage des paramètres change plus la qualité que le passage du CPU au GPU, et aucun réglage n'est bon pour toutes les fonctions : un CR faible ou une constriction aident les fonctions séparables ou unimodales d'un côté, et pénalisent Rosenbrock ou Rastrigin de l'autre. Ces essais ne changent pas les conclusions sur les temps de

TABLE XVII
EFFET DES PARAMÈTRES DU PSO ($P = 50$) : EFV MOYENNE AVEC LES PARAMÈTRES INITIAUX (INIT.) ET AVEC LA CONSTRICTION (CONSTR.).

Fonction	D	PSO-CPU		PSO-GPU	
		init.	constr.	init.	constr.
Sphere	10	0	0	0	0
	50	548	124	1101	372
	100	4096	1575	4180	2284
Rosenbrock	10	15.6	11.3	49.6	0.86
	50	4.7×10^{8}	42.5	1.5×10^{7}	5.7×10^{6}
	100	3.4×10^{8}	7.3×10^{6}	1.9×10^{8}	2.8×10^{7}
Griewank	10	0.117	0.0847	0.164	0.125
	50	12.4	9.06	16.9	8.5
	100	39.4	21.7	43.4	9.64
Rastrigin	10	1.79	8.56	2.09	8.06
	50	82.3	253	88.7	217
	100	279	630	276	587

calcul. Ils ne permettent pas non plus de reproduire les valeurs de l'article pour Rastrigin en haute dimension ; l'écart reste inexpliqué.

D. Discussion et limites

Trois facteurs expliquent les écarts de temps. (i) *Latence de lancement* : cinq kernels par génération coûtent environ 3 μs chacun, ce qui domine sur les petits problèmes ; DE-GPU-OPT n'en lance que deux, dont un recouvert. (ii) *Trafic mémoire* : DE-GPU relit et réécrit la population entre kernels, alors que DE-GPU-OPT ne la lit et ne l'écrit qu'une fois par génération. (iii) *Occupation* : DE-GPU a une configuration d'évaluation fixe (un bloc par individu), alors que DE-GPU-OPT adapte m à (P, D). Notre étude a plusieurs limites. Nous n'avons testé qu'un seul GPU (T4) et un seul thread CPU. Le calcul en double précision est lent sur la T4. Nous n'avons pas isolé l'effet de chacune des trois idées (fusion, flux, choix de m) par des expériences d'ablation, et le critère de charge de la section III-B2b n'a pas été comparé directement à la règle de l'article. Enfin, le PSO sert de repère et n'a pas été optimisé.

VI. CONCLUSION ET PERSPECTIVES

Nous avons transformé un PSO CUDA en DE/rand/1/bin et comparé trois versions de DE (CPU, CUDA de base, CUDA améliorée) ainsi que deux versions de PSO (CPU et CUDA), avec le même protocole, sur 1 800 exécutions. Quatre résultats ressortent. D'abord, la version améliorée est de $2{,}4$ à $33\times$ plus rapide que le DE séquentiel et de $1{,}35$ à $2{,}9\times$ plus rapide que la version CUDA de base ; cette dernière est même plus lente que le CPU sur le plus petit problème. Ensuite, les trois versions de DE ont des erreurs finales statistiquement indiscernables : l'accélération ne coûte pas de qualité. Le PSO-GPU, limité par son kernel d'évaluation à un thread par particule, n'atteint que $9{,}4\times$, et DE est meilleur que PSO dans la plupart des cas, sauf sur Rastrigin en grande dimension. Enfin, la qualité dépend surtout de P et des paramètres : à budget fixe, une grande population ne fait que peu de générations et réussit moins souvent.

Nos résultats concordent avec l'article de référence pour Sphere et Griewank avec $P = 50$ et $P = 100$, mais pas pour $P = 500$ ni pour Rastrigin en haute dimension ; nous n'avons pas pu en identifier la cause. Les perspectives sont : (i) lancer $P = 1000$ et 25 exécutions pour une comparaison directe ; (ii) choisir CR fonction par fonction (par exemple $CR = 0{,}1$ pour Rastrigin, section V-C6) ; (iii) faire une ablation de chacune des trois idées ; (iv) aligner le PSO-GPU (un bloc par particule) pour une comparaison plus équitable avec DE.

REMERCIEMENTS

Nous remercions Lhassane Idoumghar pour le sujet et le code CUDA de départ, ainsi que Kaggle pour l'accès au GPU.

RÉFÉRENCES

[1] A. K. Qin, F. Raimondo, Y. S. Ong, and F. Forbes, "An improved CUDA-based implementation of differential evolution on GPU," in *Proc. GECCO*, Philadelphia, USA, 2012.

[2] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.

[3] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE ICNN*, 1995, pp. 1942–1948.

[4] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE ICEC*, 1998, pp. 69–73.

[5] P. N. Suganthan *et al.*, "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Nanyang Technological University, Tech. Rep., 2005.

[6] NVIDIA, *CUDA C Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/